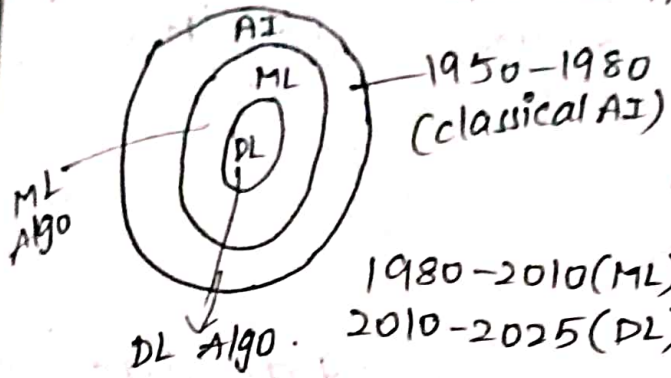


## AI

\* AI is an umbrella term.



ML - learns the logic from data we can't do handcrafting.

PAA - we can do handcrafting  
DL - Algorithm modeled after human brain.

ML/DL → learn from data / experience.

\* problems are solved by: → searching, Logic, probabilities.

## \* Searching.

① state space search.

② uninformed search / blind search - BFS, DFS.

③ Informed search / heuristic search.

## \* State space.

① Any element can represent as states / Nodes.

② We have start node / start state.

③ Goal state (where we have to reach).

④ successor function

⑤ Path

⑥ cost (optional)

Ex: ① Wolf, Goat, cabbage, Man. River crossing.

man, wolf, Goat, cabbage  
Left

limit  
2

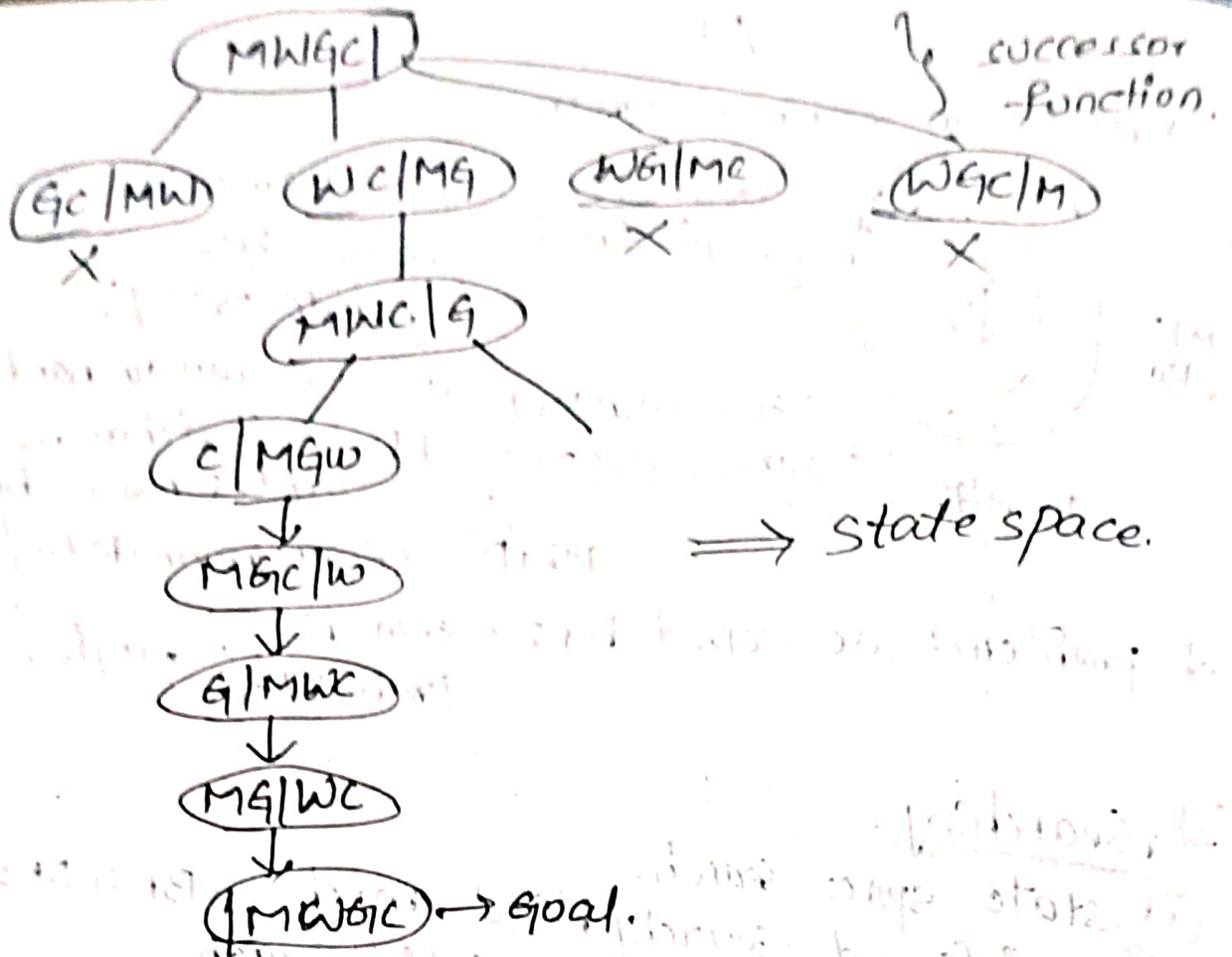
man has to drive the boat

Right.

\* state state - MWGC /

\* Goal state - /MWGC.

\* Wolf → Goat → cabbage



## ② 8 Puzzle

|   |   |   |
|---|---|---|
| 6 |   | 3 |
| 1 | 4 | 2 |
| 8 | 5 | 7 |

Initial state

By using empty space, arrange no's in order

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 8 |   | 4 |
| 7 | 6 | 5 |

Goal state

start ⇒

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 8 | 6 |   |
| 7 | 5 | 4 |

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 8 |   | 6 |
| 7 | 5 | 4 |

|   |   |   |
|---|---|---|
| 1 | 2 |   |
| 8 | 6 | 3 |
| 7 | 5 | 4 |

store them as objects

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 8 | 6 | 4 |
| 7 | 5 |   |

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 8 | 6 | 4 |
| 7 |   | 5 |

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 8 |   | 4 |
| 7 | 6 | 5 |

⇒ Goal

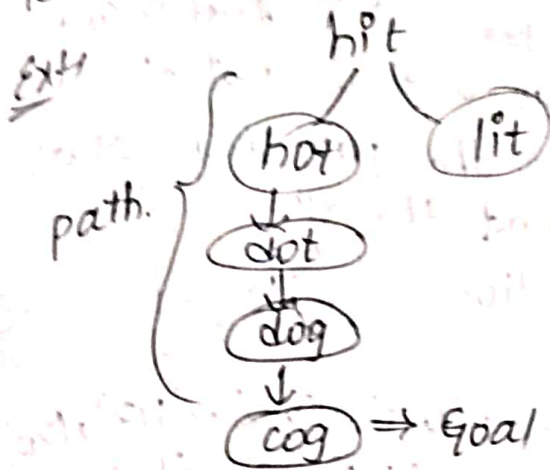
### ③ Word ladder

start-word = "hit".

end-word = "cog".

word-set = [hot, dog, dot, cog, log, lit].

\* we have st word, at a time we can change 1 letter & make it as end-word.



code:-

class State:

def \_\_init\_\_(self, word, end-word, word-set):  
 // initialize the state

self.word = word;

self.end-word = end-word;

self.word-set = word-set;

def goalTest(self):  
 // return if current state is goal test

if self.word == self.end-word:  
 return True.

else  
 return False.

def moveGen(self):  
 // generate the children of current state

children = []

for i in range(0, len(self.word)):

for c in "abcdefghijklmnopqrstuvwxyz":

new-word = self.word[:i] + c + self.word[i+1:]

if new-word in self.word-set and new-word != self.word:

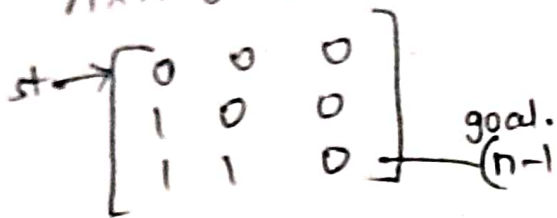


children.append(state(new\_word, self.end\_word, self.word\_set))

return children;

## A) Shortest Path Binary Matrix

$n \times n$  grid filled with 0's & 1's.



$\rightarrow$  Go from st to goal & only go with 0's not 1's.

$(n-1, n-1) \rightarrow$  Find the path from start to end through 0's.

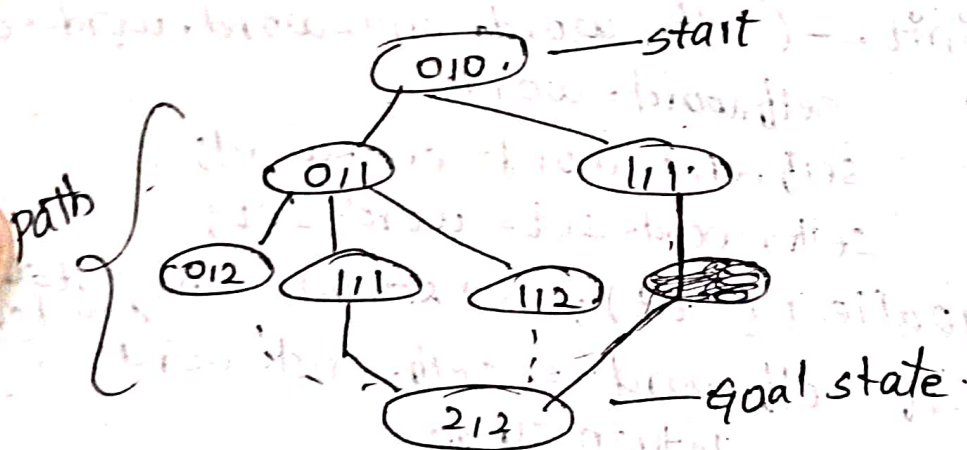
$\rightarrow$  We can move in any direction  $\begin{bmatrix} \uparrow & \uparrow & \uparrow \\ \leftarrow & \downarrow & \rightarrow \end{bmatrix}$ .



path:  $[(0,0), (0,1), (0,2), (1,2), (2,2)]$

end path:  $[(0,0), (1,1), (2,2)] \rightarrow$  shorter path.

path:  $[(0,0), (0,1), (1,1), (1,2), (2,2)]$



code: class state:

def \_\_init\_\_(self, ~~self~~ row, ~~self~~ col, ~~self~~ grid)

self.row = row

self.col = col

self.grid = grid

self.n = len(grid)

def goalTest(self):

if self.row == self.n-1 and self.col == self.n-1

return True

else

return false

def moveGen(self):

if (row > 0 and row < n):

and (col > 0 and col < n):

if (row == 0 and col == 0):

0 ≤ new-row < n.  
0 ≤ new-col < n  
grid[row][col] != 1.

children = []

directions = [(-1, -1), (-1, 0), ..., (1, 1)]

for dir in directions:

new-row = self.row + dr

new-col = self.col + dc

if 0 ≤ new-row < self.n and 0 ≤ new-col < self.n:

if self.grid[new-row][new-col] == 0:

child.obj = State(new-row, new-col, self.grid)

children.append(child.obj)

return children

① Man, Wolf, Goat, Cabbage, River Crossing.

code: class State:

def \_\_init\_\_(self, man, wolf, goat, cabbage):

self.man = man

self.wolf = wolf

self.goat = goat

self.cabbage = cabbage

def goatTest(self):

if self.man == self.wolf == self.goat == self.cabbage == "R":

return True

else

return False

```
def moveGen(self):
```

Word ladder

```
class State:
```

```
    ① def __init__(self, word, end-word, word-set):  
        self.word = word
```

```
        self.end-word = end-word
```

```
        self.word-set = word-set
```

```
    ② def goalTest(self)
```

```
        if self.word == self.end-word:
```

```
            return True
```

```
        else:
```

```
            return False
```

```
    ③ def moveGen(self):
```

```
        children = []
```

```
        for i in range(len(self.word)):
```

```
            for c in "abcd...z":
```

```
                new-word = self.word[:i] + c +
```

```
                    self.word[i+1:]
```

```
                if (new-word in self.word-set and  
                    new-word != self.word):
```

```
                    child-obj = State(new-word, self.end-  
                                     word, self.word-set)
```

```
                    children.append(child-obj)
```

```
        return children
```

```
start-word = "hit"
```

```
end-word = "cog"
```

```
word-set = ["hot", "dot", "dog", "cog", "log", "log"]
```

```
start-state = State(start-word, end-word, word-set)
```

```
print("Is goal?", start-state.goalTest())
```



for child in start-state.moveGen(): <sup>is goal? false</sup> <sub>hit</sub>  
print(child)

state-2 = State(start-word, end-word, word-set)  
state-2 = start-state

if start-state == state-2: → Equal

print("Equal")

else: print("Different") <sup>// Different</sup>

⑤ def \_\_eq\_\_(self, other): <sup>// comp the content of two obj instead of reference.</sup>  
if (self.word == other.word):  
return True

else:  
return False

if (start-state == state-2):

print("Equal")

else:  
print("Diff")

// Equal (if we have \_\_eq\_\_ method)  
o/w gives diff.

l = set()

l.add(state-2) <sup>// error.</sup>

⑥ def \_\_hash\_\_(self): <sup>// unique hashvalue for a state.</sup>  
return hash(self.word)

l = set()

l.add(state-2) <sup>// added.</sup>

l.add(start-state)

for e in l:

// print("Word:", e, "children:", e.moveGen(),  
"Goal?", e.goalTest())

print("Word:", e, "Goal?", e.goalTest())

print("children: ")

for c in e.moveGen():

print("→", c)

// Word: hit  
Goal? false

children:

→ lit

→ hot

(dup are not allowed)

② MNLGBC

class State:

def \_\_init\_\_(self, man, wolf, goat, cabbage):

self.man = man

self.wolf = wolf

self.goat = goat

self.cabbage = cabbage

} if L or R

① def goalTest(self):

if self.man == self.wolf == self.goat == self.cabbage == 'R':

return True

else:

return False

② def \_\_str\_\_(self):

return "man: " + self.man + "wolf: " + self.wolf  
+ "goat: " + self.goat + "cabbage: " + self.cabbage

③ def \_\_moveGen\_\_(self):

children = []

items = ["wolf", "goat", "cabbage", None]

new\_side = "R" if self.man == 'L' else 'L'

for e in items:

if e == "wolf" and self.man == self.wolf:

child\_obj = State(man=new\_side,  
wolf=new\_side, goat=self.goat,  
cabbage=self.cabbage)

Same for  
cabbage also  
else main only change

elif e == "goat" and self.man == self.goat:

child\_obj = State(man=new\_side, wolf=self.wolf,  
goat=new\_side, cabbage=self.cabbage)

children.append(child\_obj) → if child\_obj is a object  
return children



```

start-state = State(mane="L", wolf="L", goat="L",
                    cabbage="L")
print(start-state) // Man:L Wolf:L Goat:L Cabbage:L
print("Is Goal?" start-state.goalTest()) // False.
for c in start-state.moveGen():
    print("→", c)

```

```

④ def issafe(self):
    if self.goat == self.cabbage and self.man != self.goat:
        return False.
    elif self.wolf == self.goat and self.man != self.wolf:
        return false
    else:
        return True.

```

## \* Search

### state space search

Represent state

```

class state:
    -- init --()
    goalTest()
    moveGen()

```

## \* Uninformed Search / Blind Search.

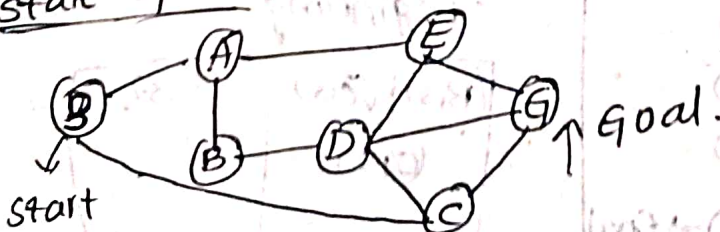
— BFS

— DFS

— IDDFS

(Iterative Deepening  
PFs)

state space:



→ we don't have any info about goal.  
→ search all states

| Node | Move Gen() | Goal Test() | If Node is goal or not.<br>(not T or F). |
|------|------------|-------------|------------------------------------------|
| S    | {A, B, C}  | F           |                                          |
| A    | {B, C, E}  | F           |                                          |
| B    | {A, D}     | F           |                                          |
| C    | {D, G}     | F           |                                          |
| D    | {B, C, G}  | F           |                                          |
| E    | {A, D, G}  | F           |                                          |
| G    | {E, D, C}  | T           |                                          |

\* Main 2 parts:

OPEN: nodes that are generated but not tested for goal.

CLOSE: nodes that are generated and tested for goal.

→ In open add nodes & if the node is not goal move it to close & generate its children & add it to open.

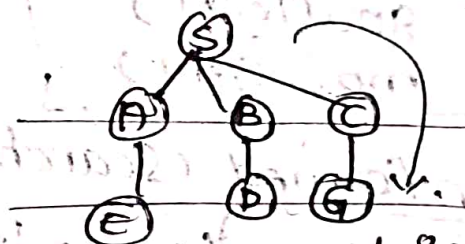
(Add new nodes only)

| OPEN (pick from beginning, parent) | CLOSED.        |
|------------------------------------|----------------|
| (S, Nil)                           | (S, Nil).      |
| (A, S), (B, S), (C, S)             | (A, S).        |
| (B, S), (C, S), (E, A)             | (B, S)         |
| (C, S), (E, A), (D, B)             | (C, S)         |
| (E, A), (D, B), (G, C)             | (E, A)         |
| (D, B), (G, C)                     | (D, B)         |
| (G, C)                             | (G, C) → Goal. |
| STOP.                              | → stop.        |

FIFO (Queue)

Path: S - C - G //

(G came from parent C.  
C came from parent S).



we are traversing level by level.  
∴ It is BFS //

DFS → LIFO (stack).

| OPEN.                          | CLOSED.       |
|--------------------------------|---------------|
| (S, Nil)                       | (S, Nil)      |
| (A, S), (B, S), (C, S)         | (A, S)        |
| (E, A), (B, S), (C, S)         | (E, A)        |
| (D, E), (G, E), (B, S), (C, S) | (D, E)        |
| (G, E), (B, S), (C, S)         | (G, E) → Goal |
|                                | → stop.       |

Now append children at beginning.

|                   |                   |                   |
|-------------------|-------------------|-------------------|
| (B, S)            | (C, S)            | (B, S)            |
| <del>(C, S)</del> | <del>(C, S)</del> | <del>(B, S)</del> |





Path:  $\rightarrow (S-A-E-D-G)$

Here we are travelling in depth.  
 $\therefore$  It is DFS.

Ex:  $\rightarrow$

| Node                 | Move Gen. |
|----------------------|-----------|
| S                    | D C B A   |
| A                    | S B J E   |
| B                    | S F A     |
| C                    | S D H G   |
| D                    | S I C     |
| E                    | A J K     |
| F                    | B K J     |
| Goal $\rightarrow$ G | C L       |
| H                    | C I M L   |
| I                    | D H       |
| J                    | A F E     |
| K                    | E F       |
| L                    | G H M     |
| M                    | L H       |

DFS:  $\rightarrow$  (LIFO)

| Open                                   | Closed   |
|----------------------------------------|----------|
| (S, Nil)                               |          |
| (D, S), (C, S), (B, S), (A, S)         | (S, Nil) |
| (I, D), (C, S), (B, S), (A, S)         | (D, S)   |
| (H, I), (C, S), (B, S), (A, S)         | (I, D)   |
| (M, H), (L, H), (C, S), (B, S), (A, S) | (H, I)   |
| (L, H), (C, S), (B, S), (A, S)         | (M, H)   |
|                                        | (L, H)   |

| Open                                                                                         | Closed |
|----------------------------------------------------------------------------------------------|--------|
| (G, L), (C, S), (B, S), (A, S)                                                               | (G, L) |
|                                                                                              | Goal   |
|                                                                                              | Stop.  |
| $\therefore$ path: $\rightarrow$                                                             |        |
| $S \rightarrow D \rightarrow I \rightarrow H \rightarrow M$<br>$\rightarrow L \rightarrow G$ |        |



# BFS. (FIFO)

| Open                                             | closed.      |
|--------------------------------------------------|--------------|
| (S, Nil)                                         | (S, Nil)     |
| (DIS)   (CIS)   (BIS)   (AIS)                    | (DIS)        |
| <del>(SID)   (TID)   (EIS)  </del>               | (CIS)        |
| (CIS)   (BIS)   (AIS), <del>(DIS)</del> , (TID)  | (BIS)        |
| (BIS)   (AIS)   (TID)   (HIC)   (GIC)            | (AIS)        |
| (AIS)   (TID)   (HIC)   (GIC),<br>(FIB)          | (TID)        |
| (TID)   (HIC)   (GIC)   (FIB),<br>(JIA)   (EIA)  | (HIC)        |
| (HIC)   (GIC)   (FIB)   (JIA)   (EIA)            | (GIC) → Goal |
| (GIC)   (FIB)   (JIA)   (EIA),<br>(M+H)   (L+H). | ↓<br>stop    |

∴ path:

S - C - G

## Pseudocode of BFS

BFS(S)

OPEN ← [(S, Nil)]

CLOSED ← []

while OPEN is not Empty:

nodepair ← OPEN[0]

node, parent ← nodepair

if goalTest(node):

return path(node, closed)

else:

CLOSED ← CLOSED + node

```

children ← move Gen (node)
new-nodes ← RemoveSeen (children, OPEN, CLOSED)
new-pairs ← makepairs (new-nodes, node)
OPEN ← OPEN + new-pairs

```

return empty list.

pseudocode of DFS.

DFS(s)

OPEN ← [(s, None)]

CLOSED ← []

while OPEN is not empty:

nodepair ← OPEN[0].

node, parent ← nodepair

if goalTest(node):

return path(node, CLOSED)

else:

CLOSED ← CLOSED + node

children ← move Gen (node)

new-nodes ← RemoveSeen (children, OPEN, CLOSED)

new-pairs ← makepairs (new-nodes, node)

OPEN ← ~~OPEN~~ new-pairs + OPEN

return empty list.

BFS python code

def bfs(start):

OPEN = [(start, None)].

CLOSED = []

while OPEN:

node-pair = OPEN.pop(0)

N, parent = node-pair

if N.goalTest():

print("Goal found")



```

path = reconstructpath(node-pair, CLOSED)
else: Path, delete() → for p in path
    Point(x, y, p)
CLOSED.append(node-pair) return
children = N.moveGen()
new-nodes = removeSeen(children, OPEN, CLOSED)
new-pairs = [(C, N) for C in new-nodes]
OPEN = OPEN + new-pairs. // for dfs
OPEN = new-pairs + OPEN
return []

```

```

def removeSeen(children, OPEN, CLOSED):
    open-nodes = [node for node, parent in OPEN]
    closed-nodes = [node for node, parent in CLOSED]
    new-nodes = [c for c in children if c not in
                  open-nodes and c not in closed-nodes]
    return new-nodes

```

```

def reconstructpath(node-pair, CLOSED):
    path = []

```

```

    parent-map = {}

```

```

    for node, parent in CLOSED:
        parent-map[node] = parent

```

```

    node, parent = node-pair

```

```

    path.append(node)

```

```

    while parent is not None:

```

```

        path.append(parent)

```

```

        parent = parent-map[parent]

```

```

    return path

```

```

start-word = "hit"

```

```

end-word = "cog"

```

```

wordset = ["hot", "dot", "dog", "cog", "log", "log"]

```

```

start-state = State(start-word, end-word, wordset)

```

```

bfs(start-state)

```

### 3) Iterative Deepening DFS

\* comparison of DFS & BFS

|                     | DFS                                                   | BFS                            |
|---------------------|-------------------------------------------------------|--------------------------------|
| Time complexity     | $O(b^d)$                                              | $O(b^d)$                       |
| Space complexity    | Linear<br>$O(bd)$                                     | Exponential<br>$O(b^d)$        |
| Quality of solution | It will find longer path. <small>to guarantee</small> | Always find the shortest path. |
| Completeness        | No Termination                                        | Terminates                     |

$b$  = branching factor  
 $d$  = depth of goal.

Guarantees the shortest path.

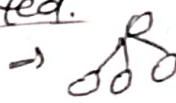
Always terminates if goal is there.

may not terminate in  $\infty$  state space.

(terminates or not?)  
T.C  $\rightarrow$  no. of nodes inspected.

$\Rightarrow$  branching factor =  $b$ .

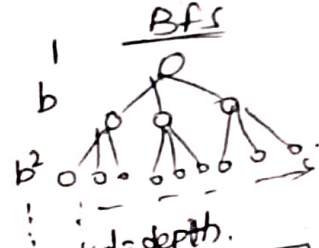
depth of the goal =  $d$ .



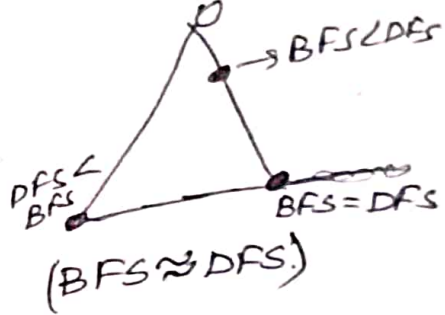
$b=3$  (each node generate 3 children)



$T.C = O(b^d)$

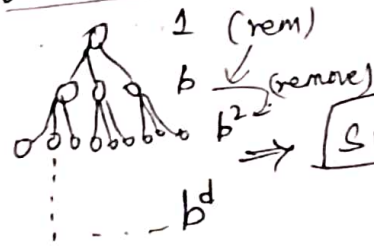


$T.C = O(b^d)$



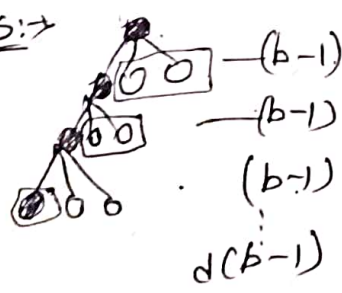
S.C  $\rightarrow$  Growth of stack/queue we maintained.

BFS  $\rightarrow$



$S.C = O(b^d)$

DFS  $\rightarrow$



$S.C = O(bd)$

\* Infinite state space  $\rightarrow$  we can count no. of nodes.

$\rightarrow$  In this BFS, DFS will terminate.

② Infinite state space  $\rightarrow$  we can't count the states (no of nodes)

$\rightarrow$  It may have many config's.

$\rightarrow$  If goal is there, BFS is always terminates. DFS is not terminated, if it select x path.

\* so we can combine 2 Algorithms, to get optimized Algorithm.

(3) Iterative Deepening DFS. (IDDFS / DFID).

DFS  $\rightarrow$  Apply limit / bound.

$db = 1 \Rightarrow$  apply dfs in 1 level.

  $\Rightarrow$  If goal is not found,  $db++$

$db = 2.$



$db = 3$

$\rightarrow db++$

If goal is found, then terminate it.

$\rightarrow$  As we are finding in a level, we can find shortest path.

$\rightarrow$  But in level 2, we are searching level 1 again. (which is ~~disadv~~)



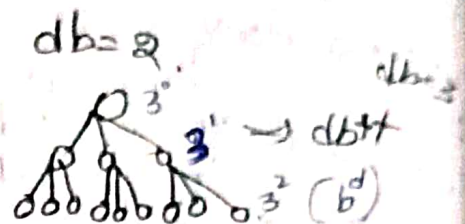
\* some can combine 2 Algorithms, to get optimized algorithm.

### (3) Iterative Deepening DFS (IDDFS/IDTD).

DFS  $\rightarrow$  Apply limit/bound.

db=1  $\Rightarrow$  apply dfs in 1 level.

  $\Rightarrow$  If goal is not found, db++



If goal is found, then terminate it.

$\rightarrow$  As we are finding in a level, we can find shortest path.

$\rightarrow$  But in level 2, we are searching level 1 again. (which is desirable)

\* Here  $\rightarrow$  Applying Depth bound DFS by iteratively increasing the depth.

db=0  $\Rightarrow$    $\rightarrow$  check it is goal or not & if goal is found break or else restart algo by db++.

db=1  $\Rightarrow$    $\Rightarrow$  generate children (db=1) & apply dfs  $\Rightarrow$  If goal not found, then db++.

\* Like this we are applying dfs by applying bounds.

\* It helps to apply dfs by using less memory & helps to find shortest path.

(space comp = linear) // & T.C  $\sim$  No. of nodes inspected

$$T.C = 1 \times (d+1) + b^1 \times d + b^2 (d-1) + b^3 (d-2) \dots b^{d-1} (2) + b^d (1)$$

$$T.C = O(b^d)$$

b = branching factor (no of children)

d = depth of goal.

\* It gives shortest path, takes linear space & takes same T.C as bfs, dfs.

(N, parent, depth)

$\downarrow$   
children

(C, N, depth+1)

$\rightarrow$  (check in github for algorithm)

# \* Informed Search / Heuristic Search.

Heureka → Eureka (I know)

- \* In uninformed srch, we have to find by searching.
- \* we have state repres<sup>n</sup> & we have to search for goal.
- \* In informed srch, if some steps are taking us away from the goal, then discard it, o/w go with that step.

Ex: 8-puzzle problem

|   |   |   |
|---|---|---|
| 6 |   | 7 |
| 5 | 3 | 4 |
| 1 | 2 | 8 |

Initial state.

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 |   |

Goal state

\* Total possible states =  $9!$

\* Total no. of reachable states =  $\frac{9!}{2} = 181,440$ .

\* It is difficult for blind srch. (b/c it generates so many states).

↓ so we have to step close to goal state which is similar to goal.

files. ↙

|   |   |   |
|---|---|---|
| 6 |   | 7 |
| 5 | 3 | 4 |
| 1 | 2 | 8 |

$S_1$

|   |   |   |
|---|---|---|
| 1 | 3 |   |
| 4 | 5 | 6 |
| 7 | 8 | 2 |

$S_2$

⇒ It is nearer to goal, so we need to go with this.

$h_1$  = no. of tiles not in position. Heuristic →

$$h_1(S_1) = 9.$$

$h_2(S_2) = 3$  ⇒ so we can go with this.

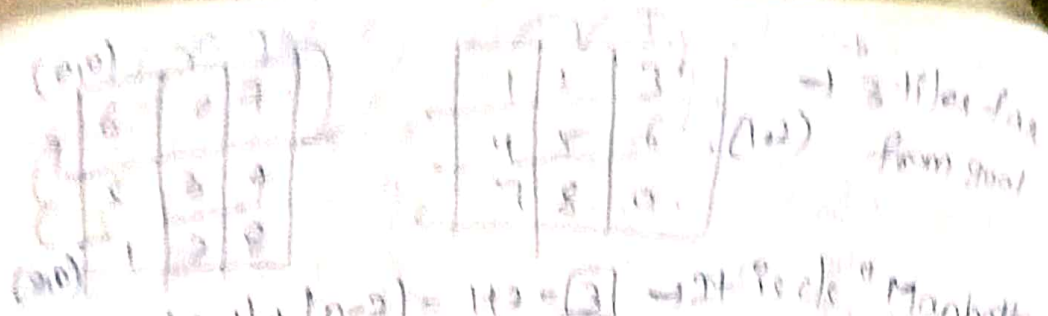
\* Heuristic srch helps us to find how close/far from the goal.

\* It doesn't guarantee sol<sup>n</sup>, but it will help us to find quickly.

\* we can also find the dist of tile from original pos<sup>n</sup>. (If dist is less, we go with that)

$h_2$  = Manhattan distance of the tiles from current pos<sup>n</sup> to goal pos<sup>n</sup>.





dis =  $|0-1| + |0-2| = 1+2 = 3 \rightarrow 3$  tiles far from goal  
 for 2 =  $|2-0| + |0-0| = 2 \rightarrow 2$  tiles far from goal.

\* There are 2 algorithms.

1. Best First Search.
2. A\* Search (A star search).

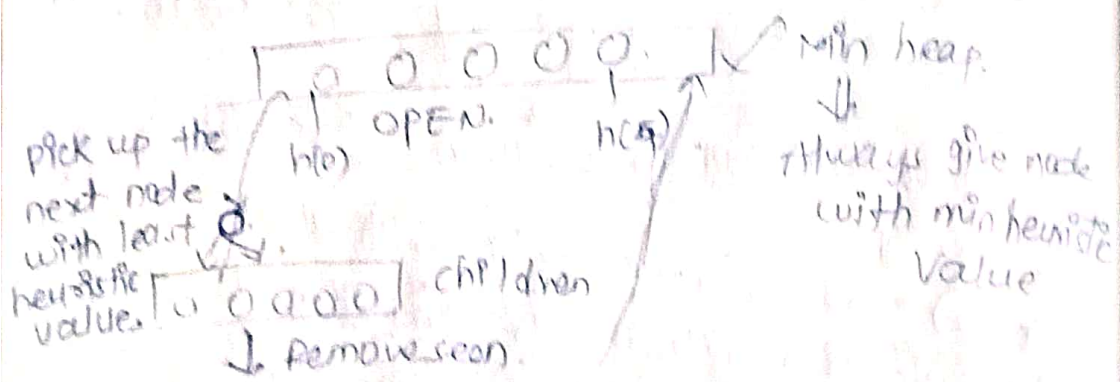
\* Heuristic  $f^n$  = estimated cost to the goal.

3-puzzle

$h_1$  = no. of tiles not in position.  
 $h_2$  = Manhattan distance of the tiles from current pos<sup>n</sup> to goal position.

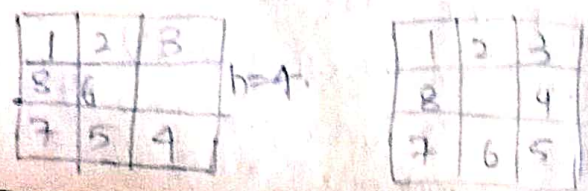
\* Best First Search

\* for every node we calculate heuristic value.  
 \* Instead of picking from LIFO/FIFO, we pick the node which has "less heuristic value".



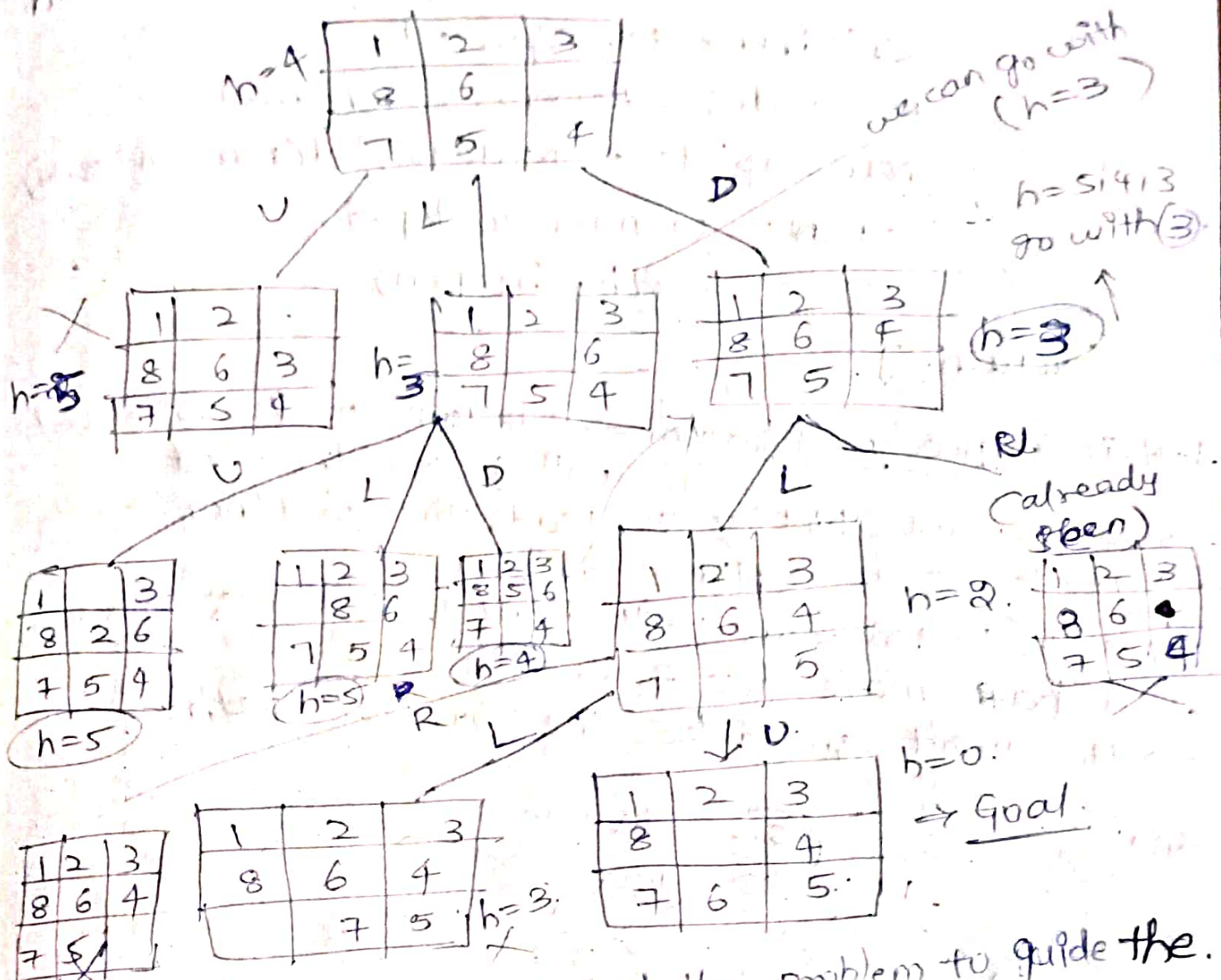
New node [0 0 0 0]

ex: 3-puzzle.





$h =$  no. of tiles not in position.




  
 (already seen)
   
 & we are using info. around the problem to guide the search.

\* Algorithm.

Best First search(start):

open  $\leftarrow [(start, nil, h(start))]$

closed  $\leftarrow [ ]$

while open is not empty:

node-triplet  $\leftarrow$  open.pop(0).

$N$ , parent, h-value ~~node-triplet~~.

if GoatTest(N):

```
goalTest(N):  
return reconstructPath(node-triplet,  
                        closed)
```

else:

closed  $\leftarrow$  closed + node-triplet

children  $\leftarrow$  MoveGen (n)

new-nodes  $\leftarrow$  removeSeen(children)

new-triplets  $\leftarrow$  MakeTriplet(new-nodes, n)

open  $\leftarrow$  open + new-triplets

open  $\leftarrow$  sortBy-h(open).

return()

\*It is a greedy algorithm  $\Rightarrow$  we look locally optimized soln & consider that.

$\rightarrow$  we are not thinking it leads to goal/not

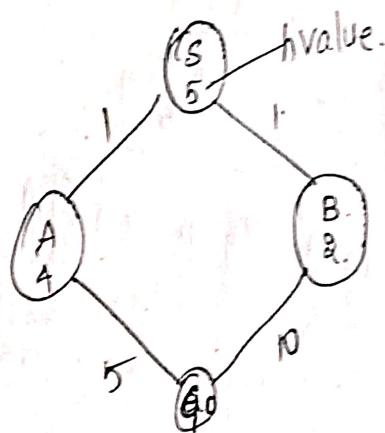
$\rightarrow$  It is not an optimal soln.

\*Greedy (person who don't think abt future consequences)

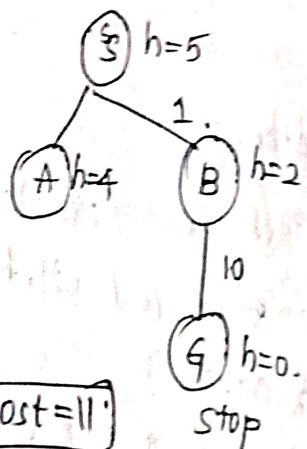
$\rightarrow$  It has more T.C & S.C.

$\rightarrow$  It doesn't guarantee the shortest path.

Ex:-



By heuristic search.  $\Rightarrow$



which is not optimized //

\* so we go with A\* Algorithm to find best

Part:

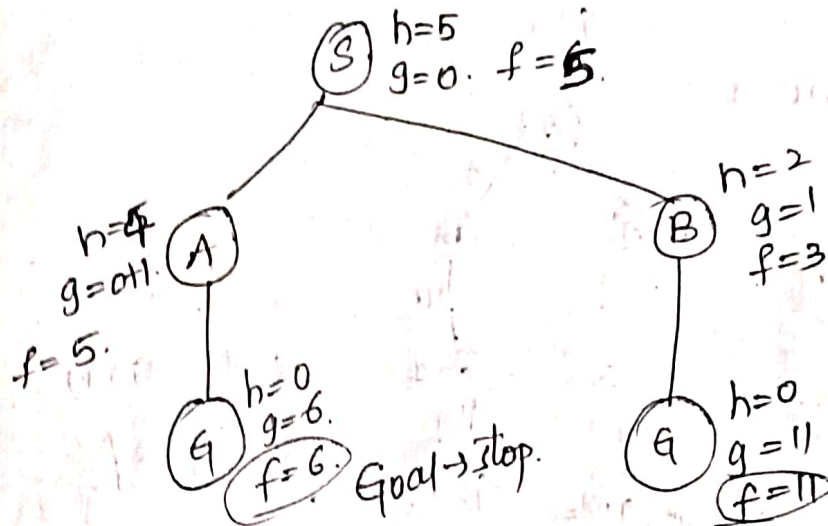


if we go to future, but we need to store past value, so that we can find optimal sol<sup>n</sup>.

$h(n)$  = estimated cost from  $n$  to goal.

$g(n)$  = actual cost from start to  $n$ .

$f(n) = g(n) + h(n)$   $\Rightarrow$  generate children based on  $f(n)$ .



Now comp  $f$  i.e.  $f=6$  through A &  $f=11$  through B.

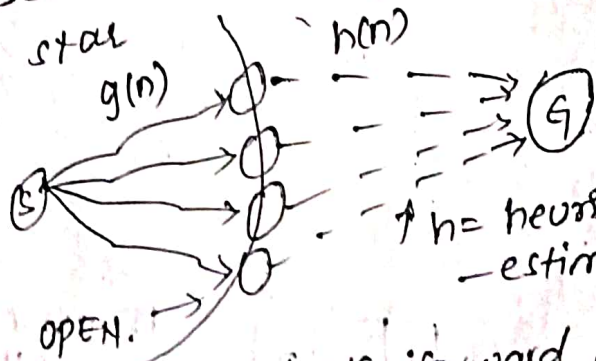
$\therefore$  parent of  $G = A$  (updated).

$\therefore$  path  $\Rightarrow$  S-A-G cost = 6. //

### \* Heuristic Search

$\rightarrow$  Best first - greedy algorithm.  $\Rightarrow$  may not find the shortest path.

$\rightarrow$  A star



$\uparrow$   $h$  = heuristic - estimated distance to goal

\* so, we need to look forward & backward.

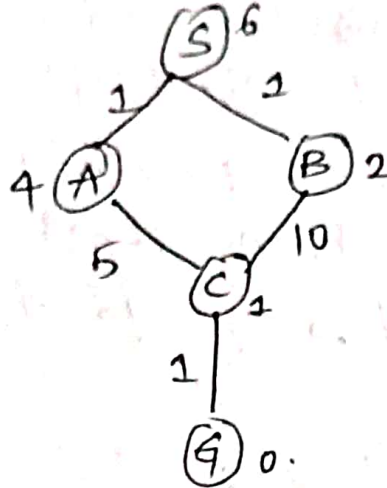
$\therefore f(n) = g(n) + h(n)$

$g(n)$  = Actual cost

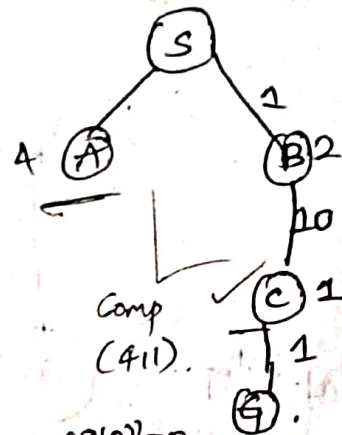
$h(n)$  = heuristic cost.



Ex-1 8-puzzle



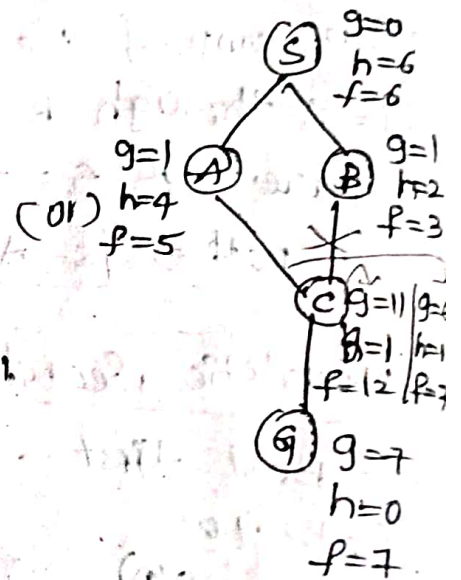
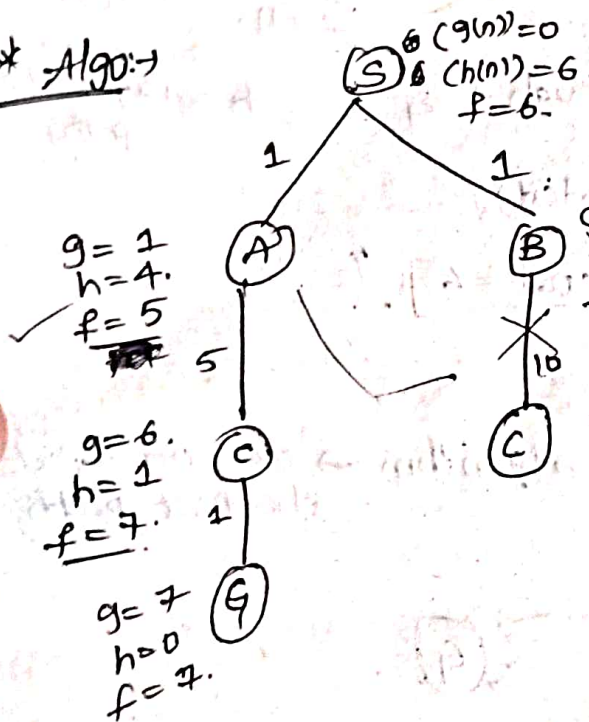
Best first src:-



$$\text{Cost} = 1 + 10 + 1$$

$$= 12$$

A\* Algo:-



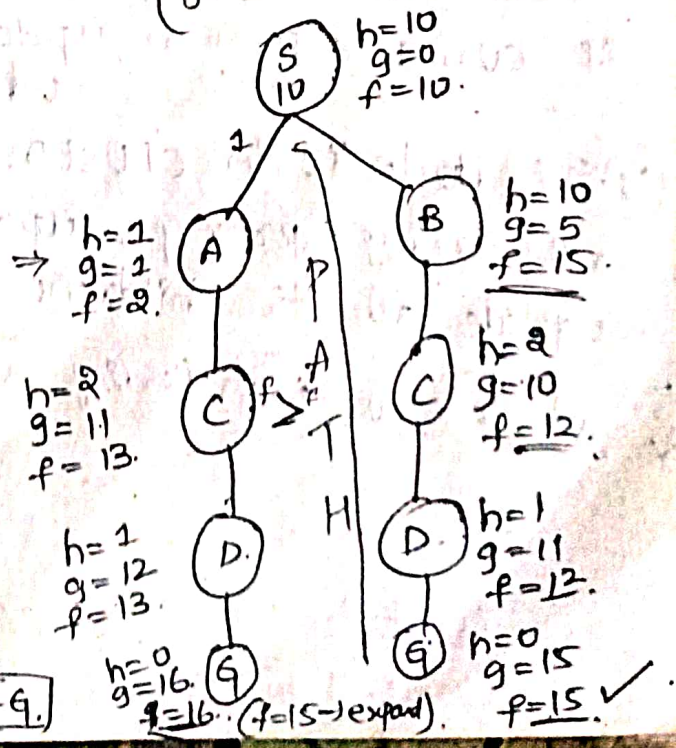
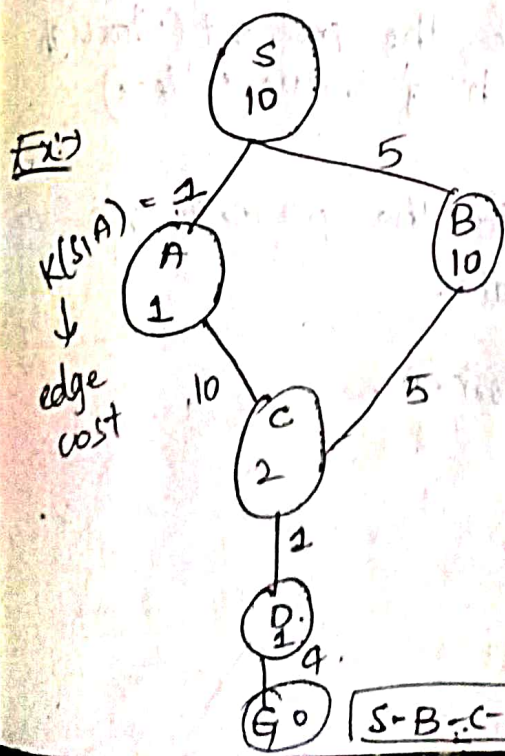
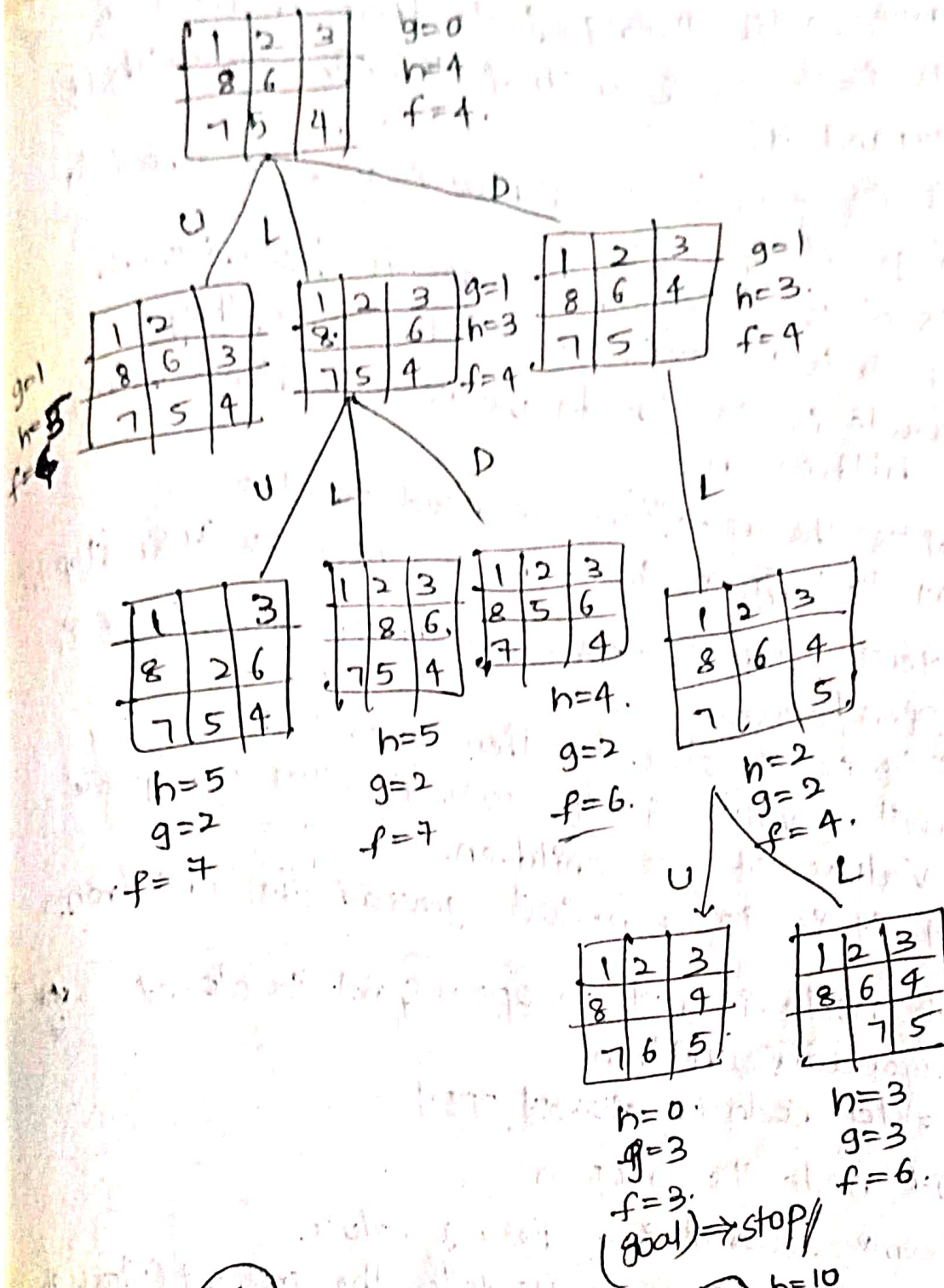
Ex-2 8-puzzle

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 8 | 6 |   |
| 7 | 5 | 4 |

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 8 |   | 4 |
| 7 | 6 | 5 |

h = no. of unmatch tiles.

g = no. of moves.





\* Here after finding goal, there is node (B) with  $f=15$  & (G) with  $f=16$ , so, go with (B) & expand it.

\* For (C) prev  $f=13$  & new  $f=12$ , so we need to change parent of C. (to B)

\* As 'c' is in closed, it means it has children.

If node is in open, no need to do anything.

If node is in closed, we need to change values of children also.

\* After the updation, we need to stop

\* At last (G) is in open, we pick up it & stop the

⇒ start with (S); when we pick, check if it is in open/closed

→ If it is in closed, then compare prev  $g$  & current  $g$  value & if prev  $g >$  curr  $g$ , update the values of its children.

→ If it is in open, just generate the children.

① If node is not in OPEN & not in closed

→ Generate children

→ Add node to ~~OPEN~~ OPEN.

② If node is in open.

→ comp current  $g$  & prev  $g$  values.

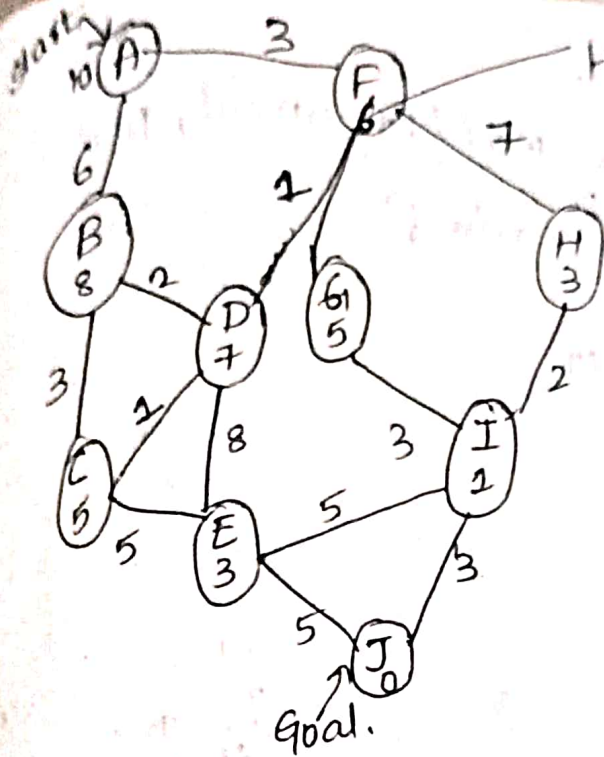
if curr  $g <$  prev  $g$ , update the parent (through which  $g$  value is less)

③ If node is in CLOSED.

→ If curr  $g <$  prev  $g$ , update the parent & update all its children's data.

\* A\* is good path finding algorithm.

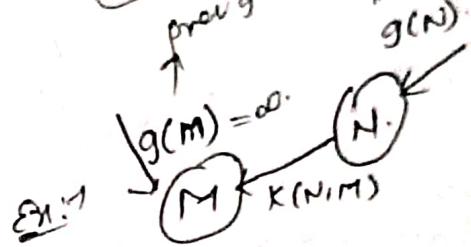




Heuristic value.

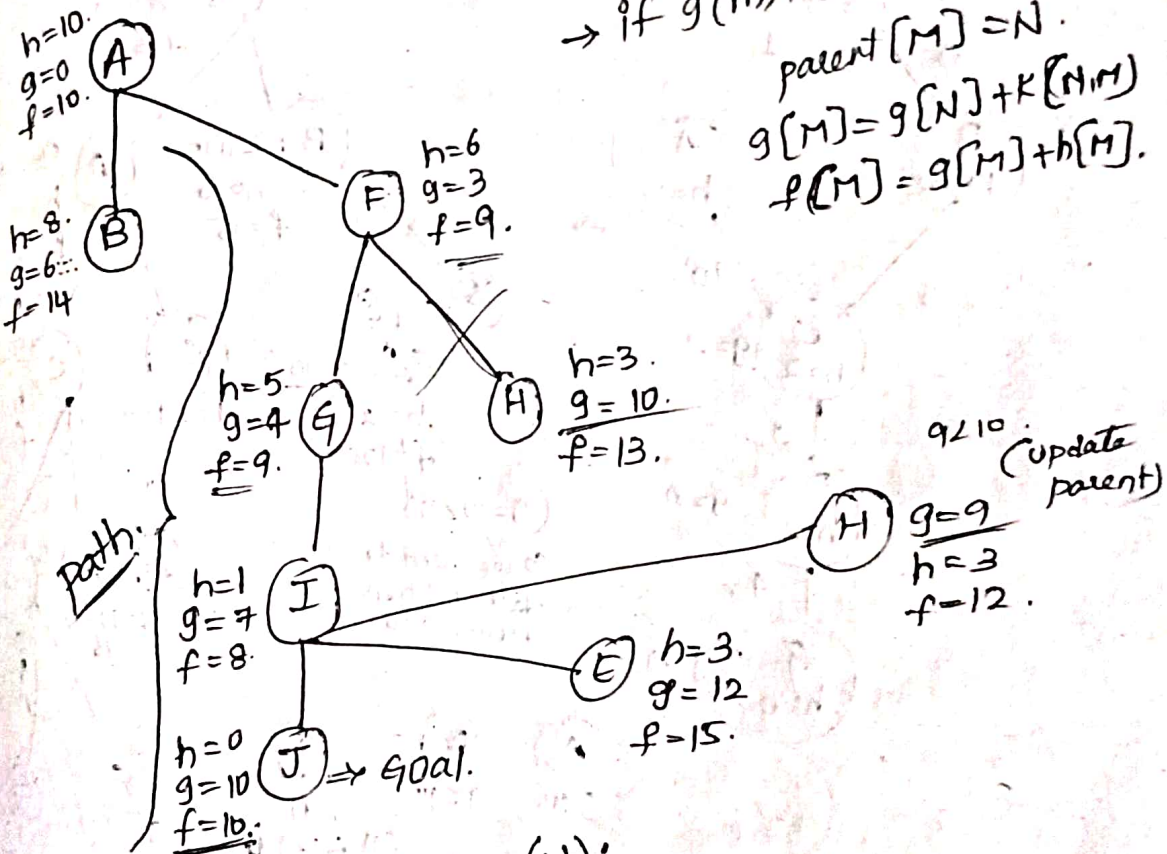
$$f(n) = g(n) + h(n)$$

If there is no path to  $M$  (prev path) then put  $\infty$ .



→ if  $g(m) > g(n) + K(N,M)$ .

$$\begin{aligned} \text{parent}[M] &= N \\ g[M] &= g[N] + K[N,M] \\ f[M] &= g[M] + h[M]. \end{aligned}$$



for  $M$  in  $\text{MoveGen}(N)$ :  
 if  $g(M) > g(N) + K(N,M)$ :  
 $\text{parent}[M] = N$   
 $g(M) = g(N) + K(N,M)$   
 $f(M) = g(M) + h(M)$   
 if  $M$  in open:  
 pass  
 # update parent.

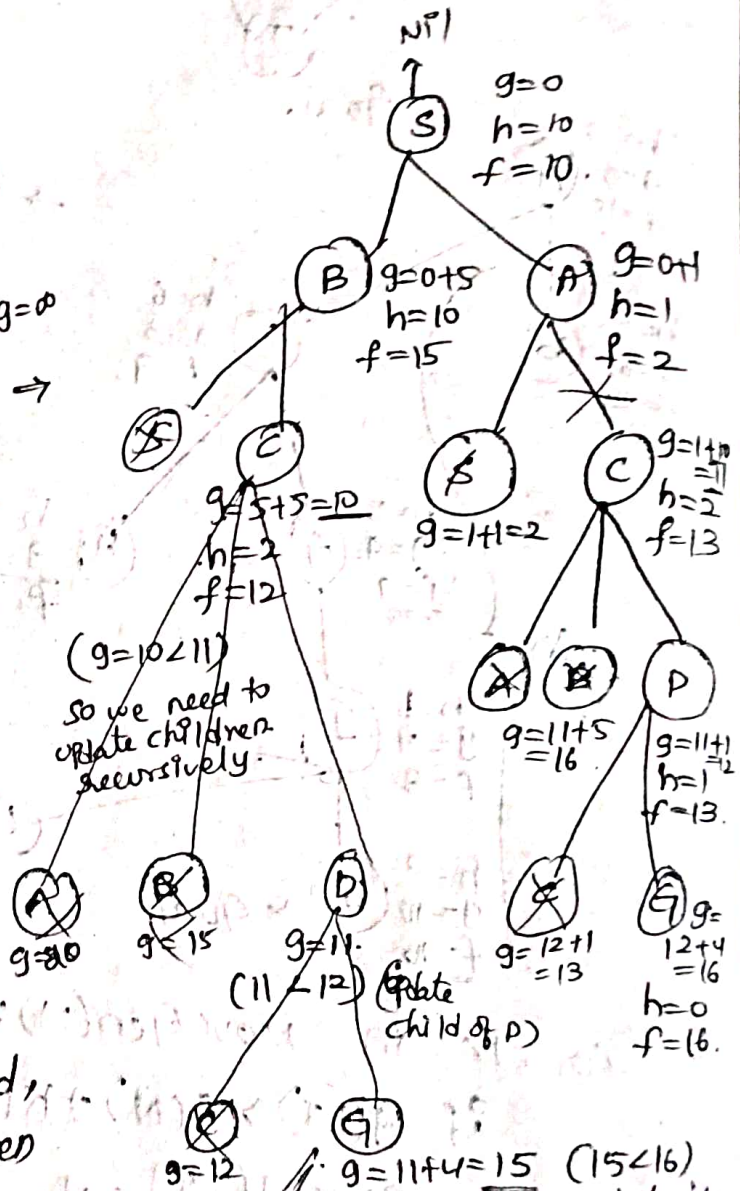
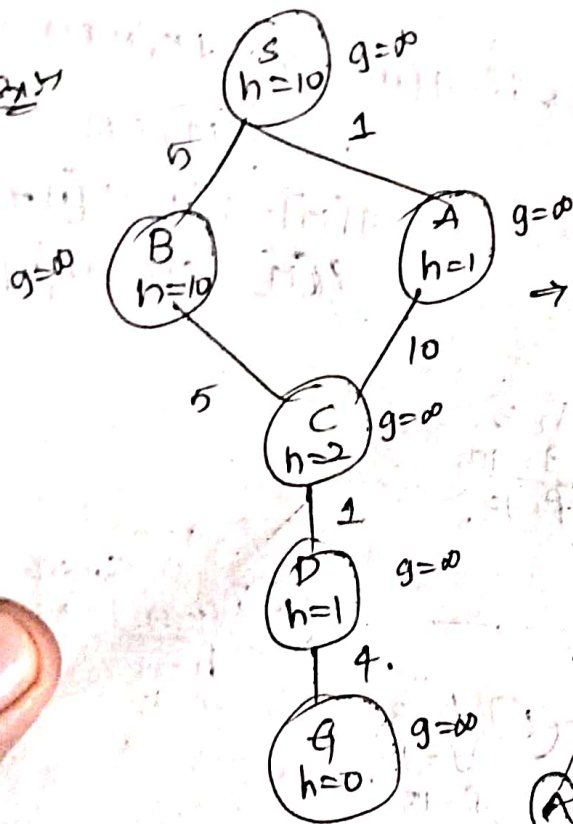
elif M in CLOSED:

propagateImprovement(M, parent, g, h)  
 $\downarrow$   
 recursive f?

else

open = open + M

Ex 1



→ If node is in closed,  
 it means it has children  
 so comp prev g & curr g  
 if  $prev\ g \neq curr\ g$ , update parent & stop  
 check if children is in closed,  
 then go again & generate its  
 children, until its children is in open  
 b/c if it is in open, that means it don't have  
 children, so just update parent & stop.



# Algorithm

Astar-search (start):

open  $\leftarrow$  [start]

closed  $\leftarrow$  [ ]

parent[start] = Nil

g  $\leftarrow$  Initialize to  $\infty$  for all nodes.

g[start] = 0

f[start] = g[start] + h(start) // calculate h value

while open is not empty:

N  $\leftarrow$  pick the min f value node from open. <sub>remove</sub>

if (GoalTest(N)):

return constructpath(N, parent, closed)

else:

closed  $\leftarrow$  closed + N

for M in MoveGen(N):

if g[M] > g[N] + K(N, M)

parent[M]  $\leftarrow$  N.

g[M]  $\leftarrow$  g[N] + K(N, M).

f[M]  $\leftarrow$  g[M] + h(M)

if M in open:

continue

else if M in closed:

propagateimprovement(M,  
parent, g, f, closed)

else

open  $\leftarrow$  open + M

return [ ]

def propagateimprovement (M, parent, g, f, closed):

for x in MoveGen(M):

if g[x] > g[M] + K(M, x):

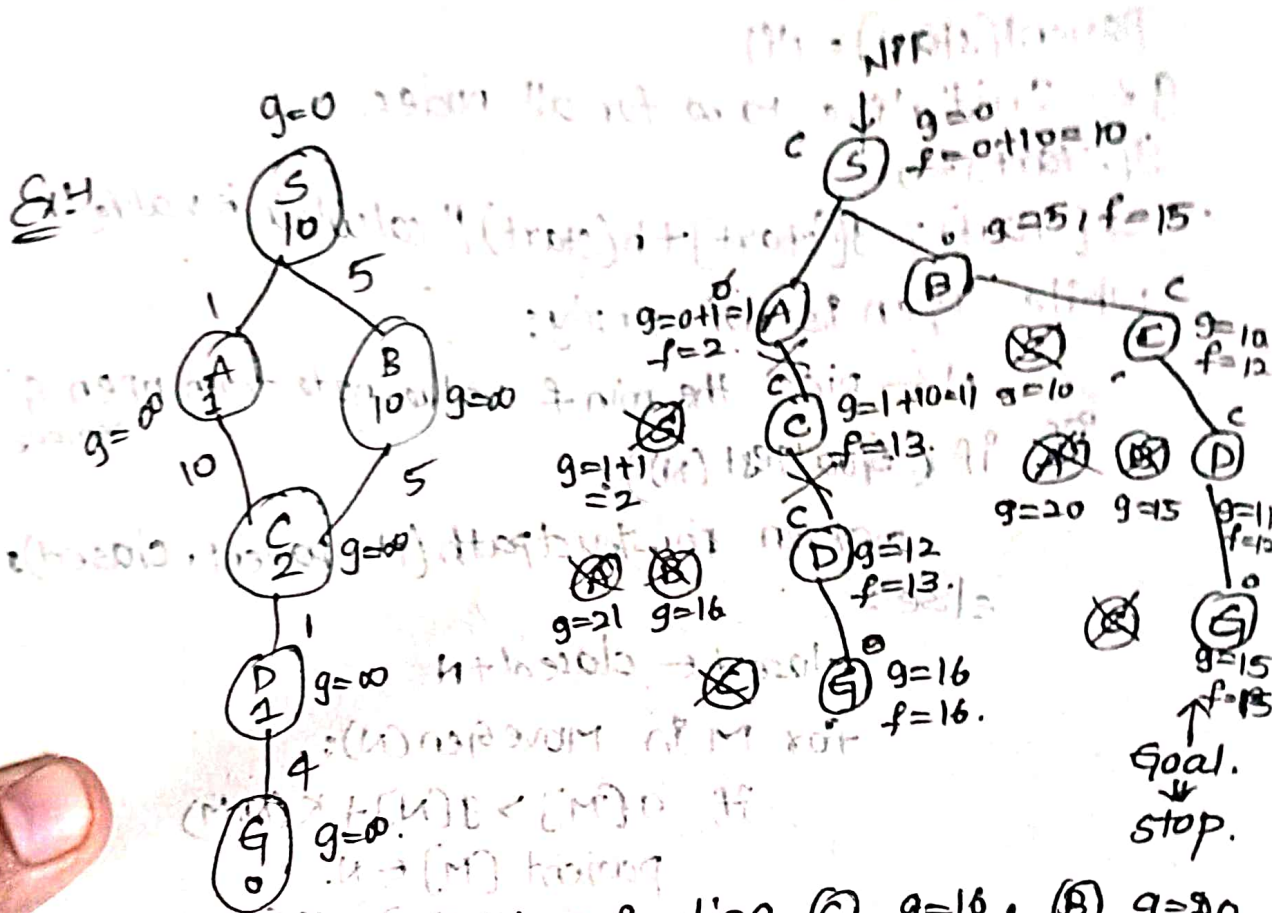
parent[x]  $\leftarrow$  M

g[x]  $\leftarrow$  g[M] + K(M, x)

$$f(x) = g(x) + h(x)$$

if  $x$  is closed:

propagate improvement ( $X$ , parent,  $g$ ,  $f$ , closed)



At the  $h(B) = 20$ . After finding (A)  $g=16$ , (B)  $g=20$   
 $f=\underline{16}$   $f=\underline{25}$   
 we pick (A)

so the path from (B) is discarded.

So the path from (B) is discarded.  
\* If  $A^*$  needs to find shortest path, the heuristic value should be in under estimate the actual cost value.  $A^*$  will find optimal path.

\* Admissibility:  $A^*$  will find optimal path only if the heuristic always under estimate the actual cost.

actual cost:  $h(n) \leq h^*(n)$

heuristic path cost from n to g. : optimal path cost from n to g.

$B \rightarrow G: 10 < h=0 \Rightarrow$  It doesn't give shortest path. X  
 $B \rightarrow G: 10 \geq h=10 \checkmark$



$$S-G: 15 \geq h=10 \checkmark$$

$$B-G: 10 \geq h=20 \times \text{ doesn't give shortest path}$$

$$A-G: 15 \geq h=1 \checkmark$$

$$C-G: 5 \geq h=2 \checkmark \quad (h \leq h^*(n)) \quad (Q \leq 5) \checkmark$$

$$D-G: 4 \geq h=1 \checkmark$$

for all nodes the cond<sup>n</sup> is true, then ~~only~~ we find guaranteed optimal path, o/w we won't find optimal path.

Ex: 8-puzzle

$h$  = no of tiles not in place. (doesn't consider space)

|   |                    |   |
|---|--------------------|---|
| 1 | 2                  | 3 |
| 8 | 7 <sup>(1,2)</sup> |   |
| 4 | 6                  | 5 |

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 8 |   | 4 |
| 7 | 6 | 5 |

$$h_1 = 2$$

actual cost = no. of moves

we can't move atleast 2 or more. to reach to the goal test  $\therefore (h \leq h^*(n))$

$\therefore h_1$  allows underestimate the <sup>actual</sup> moves  $\therefore h_1$  is admissible heuristic.

$h_2$  = Manhattan distance. of each tile from current state pos<sup>n</sup> to goal state pos<sup>n</sup>.

Ex:

|   |   |   |
|---|---|---|
| 1 | 2 | 6 |
| 8 | 7 |   |
| 4 | 3 | 5 |

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 8 |   | 4 |
| 7 | 6 | 5 |

$$h_2 = 0 + 0 + 3 + 0 + 2 + 3 + 3 + 0 = 11. \quad (\text{min 11 moves are req})$$

(Any time we move at least we need 11 moves req)

$\therefore h_2$  is admissible heuristic.